1.3.6 Test Functions

Show Lecture Video

Bugs are errors in your code. As you may already know, and certainly will experience soon, the hardest part of programming is debugging, or finding and fixing bugs. You will often spend more time debugging than actually writing code. Thus, anything we can do to help with debugging is worth the effort.

Test functions are one of the best ways to help with debugging. Here is an example:

```
1  def onesDigit(n):
2      return n%10
3
4  def testOnesDigit():
5      print("Testing onesDigit()...", end="")
6      assert(onesDigit(5) == 5)
7      assert(onesDigit(123) == 3)
8      assert(onesDigit(100) == 0)
9      assert(onesDigit(999) == 9)
10     print("Passed!")
11
12 testOnesDigit()
```

Run

This example defines the function `onesDigit()`, and then the test function `testOnesDigit()`. The sole purpose of `testOnesDigit()` is, as its name implies, to test that `onesDigit()` works properly.

A test function includes **test cases**. Each test case uses an `assert` statement to verify that the function returns the expected result for specific arguments. If that is true, the `assert` statement does nothing. But if it is false, that is, if the function returned the wrong result for that case, then the `assert` statement crashes with an **Assertion Error**.

It is very important that you include a thoughtful set of test cases. Otherwise, you may think the function works when in fact it does not. That is what happened in the test function above! Can you see the error? Maybe not. It's hard to find.

To shed light on the error, let's add one more case to our test function:

```
def onesDigit(n):
  return n%10

def testOnesDigit():
    print("Testing onesDigit()...", end="")
    assert(onesDigit(5) == 5)
    assert(onesDigit(123) == 3)
    assert(onesDigit(100) == 0)
    assert(onesDigit(999) == 9)
    assert(onesDigit(-123) == 3) # we just added
    print("Passed!")

testOnesDigit()
```

Run

This time, the test function crashes. That may seem bad, but it's not. If we have a bug (and we do!), we want the test function to crash, and therefore let us know we have an error.

Run the code again, and look carefully at the test function's error. It tells us which `assert` failed. Specifically, `onesDigit(-123)` did not return `3`. Oh no! But it is good that we now know that we have an error.

Now what? The first thing to do is to find out what `onesDigit(-123)` returned. For that, first run the code. Then, in the console, enter `onesDigit(-123)`. See the result? It is `7`, and not `3`.

This does not tell us *why* that happened. We have to figure that out now. But at least we know where to look for the bug.

As it happens, `%` behaves in surprising fashion for negative numbers. For example, `-3 % 10` is `7` and not `3`. However, since the ones digit of `-123` is the same as the ones digit for `123`, we can simply take the absolute value of the argument to avoid this problem.

Here is the fixed code:

```
def onesDigit(n):
    return abs(n) % 10 # fixed, by using the abs

def testOnesDigit():
    print("Testing onesDigit()...", end="")
    assert(onesDigit(5) == 5)
```

```
    assert(onesDigit(123) == 3)
    assert(onesDigit(100) == 0)
    assert(onesDigit(999) == 9)
    assert(onesDigit(-123) == 3)
    print("Passed!")

testOnesDigit()
```

Run

Run the code and see that it passes all the tests. Great! This still does not guarantee that the code works for every possible input, but if we have a good set of test cases, we can have high confidence that our function works properly.

@testFunction

Python's `assert` is very handy for test cases, but it has a problem. The way we are using it, the `assert` tells you the test case that failed, and the result you should have gotten. But it does not tell you the incorrect result that your code returned. That's why we have to enter `onesDigit(-123)` into the console in the example above.

To address this, we included `@testFunction` in the CPCS Utils module. To use it, just enter `@testFunction` in the line prior to your test function, like so:

```
from cmu_cpcs_utils import testFunction

def onesDigit(n):
    return n % 10 # broken again (on purpose)

@testFunction
def testOnesDigit():
    assert(onesDigit(5) == 5)
    assert(onesDigit(123) == 3)
    assert(onesDigit(100) == 0)
    assert(onesDigit(999) == 9)
    assert(onesDigit(-123) == 3)

testOnesDigit()
```

Run

Run the code and see how it provides more helpful information when the `assert` fails. Also, as a convenience, `@testFunction` prints "Running testOnesDigit()..." and "Passed!" for us, so we do not have to do that, either. This is very helpful, so we will use `@testFunction` in the test functions we provide in the exercises.

> For the curious, the `@` before `@testFunction` makes it a **function decorator**. This lets `@testFunction` modify the behavior of the function defined on the next line. We may cover function decorators later in the course. We only mention it here in case you were wondering about it.

Checkpoint 1

What will happen in the previous example if we remove `@testFunction` ?

- ○ **The tests will display an error, but it will not print the incorrect result the code returned.**
- ○ **The tests will display an error, and it will print the incorrect result the code returned.**
- ○ **The tests will not display an error, and it will print that the tests passed.**
- ○ **The tests will not display an error. Nothing will be printed at all.**

Submit

Continue